ParkingHub

Código completo explicado
index.html, style.css e script.js

Equipe

Paulo Henrique Alves de Almeida

Matheus Avanzo dos Santos

Murilo Roselini

Petterson Augusto Papa de Souza

Raphael Souza Araujo Barbosa

Cada trecho de código aparece exatamente como está no projeto, com o número das linhas, seguido da explicação do que ele faz.

Antes do código: como tudo se conecta

O ParkingHub é um site feito só com três arquivos. Cada um tem um papel:

Arquivo	Papel	Analogia
`index.html`	Estrutura: quais elementos existem na página.	O esqueleto e as paredes de uma casa.
`style.css`	Aparência: cores, tamanhos, posições.	A pintura e a decoração.
`script.js`	Comportamento: regras, cálculos, validações.	A parte elétrica: faz as coisas funcionarem.

Como se ligam: o HTML dá um `id` a cada elemento importante (ex.: `id="nome"`). O JavaScript encontra o elemento por esse id (`$("nome")`), lê ou altera seu conteúdo e liga e desliga classes e atributos. O CSS reage a essas classes e atributos (ex.: `.spot.sel` fica laranja).

Ordem recomendada de explicação

Primeiro o HTML (o que existe), depois o CSS (como parece) e por último o JavaScript (como funciona). No JavaScript, siga a ordem do próprio arquivo: ele está organizado como o usuário usa o site.

Glossário rápido

Termo	Significado
Variável (`const` / `let`)	Um nome que guarda um valor. `const` não pode ser reatribuída; `let` pode.
Função	Um bloco de código com nome, que pode ser chamado várias vezes.
Objeto	Um conjunto de propriedades: `{ nome: "Ana", placa: "ABC1D23" }`.
Array	Uma lista de itens: `[1, 2, 3]`.
Evento	Algo que acontece na página (clique, digitação, envio). `onclick`, `oninput`, `onsubmit` dizem o que fazer quando acontece.
DOM	A página vista pelo JavaScript como uma árvore de elementos que pode ser lida e alterada.
Condição (`if`)	Executa um trecho só se algo for verdadeiro.
Laço (`for`)	Repete um trecho várias vezes, por exemplo para passar por todas as vagas.
localStorage	Pequeno armazenamento do navegador: guarda texto e sobrevive ao recarregar a página.
JSON	Formato de texto para representar objetos e listas (`JSON.stringify` converte para texto; `JSON.parse` volta para objetos).
Milissegundos	Unidade de tempo que o JavaScript usa para datas: 1 hora = 3.600.000 ms.

Parte 1 - index.html

Cabeçalho do documento (head) (linhas 1 a 13)

```
 1  <!doctype html>
 2  <!-- ParkingHub - estrutura da página (versão simples).
 3       O HTML diz O QUE existe na página. Cada elemento importante tem um "id", que o JavaScript usa para
achá-lo.
 4       O atributo "class" serve para o CSS dar o visual. O atributo "hidden" esconde o elemento até o
JavaScript mostrá-lo. -->
 5  <html lang="pt-BR">
 6  <head>
 7    <meta charset="UTF-8">                                          <!-- permite acentos e
caracteres especiais -->
 8    <meta name="viewport" content="width=device-width, initial-scale=1.0">  <!-- ajusta a página à largura
do celular -->
 9    <title>ParkingHub · Reserva de vagas</title>                    <!-- texto da aba do navegador
-->
10    <link rel="icon" href="image/logo-provisoria.jpeg">                <!-- ícone da aba -->
11    <link rel="stylesheet" href="style.css">                        <!-- liga o arquivo de
aparência -->
12    <script src="script.js" defer></script>                         <!-- liga a lógica; "defer"
espera a página carregar antes de rodar -->
13  </head>
```

- `<!doctype html>` avisa ao navegador que é HTML moderno (HTML5).
- `lang="pt-BR"` informa o idioma da página (ajuda leitores de tela e corretor ortográfico).
- `<meta charset="UTF-8">` permite acentos e caracteres especiais.
- `<meta name="viewport">` faz a página se ajustar à largura do celular.
- `<title>` é o texto da aba do navegador.
- `<link rel="icon">` define o ícone da aba. `<link rel="stylesheet">` liga o arquivo `style.css`.
- `<script src="script.js" defer>` liga o JavaScript. O `defer` faz o navegador só executá-lo **depois** de ler a página inteira, para que os elementos já existam quando o JS for procurá-los.

Filtro da logo e cabeçalho da página (linhas 14 a 39)

```
14  <body>
15    <!-- Filtro invisível: no tema claro, troca o preto da logo por branco (o CSS usa o id "logo-claro") -
->
16    <svg width="0" height="0" style="position:absolute"><filter id="logo-claro" color-interpolation-
filters="sRGB"><feColorMatrix type="matrix" values="1 0 -1 0 0  0 1 -1 0 0  0 0 0 0 0  1 0 0 0 0"/></filter>
</svg>
17
18    <!-- Biblioteca de ícones (invisível): cada <symbol> é um desenho que o resto da página reutiliza com
<use href="#icone-...">.
19         Os ícones usam a cor do texto onde forem colocados. -->
20    <svg width="0" height="0" style="position:absolute" aria-hidden="true">
21      <symbol id="icone-lua" viewBox="0 0 24 24"><path d="M20.8 13.1A9 9 0 0 1 10.9 3.2 9 9 0 1 0 20.8
13.1Z"/></symbol>
22      <symbol id="icone-sol" viewBox="0 0 24 24"><circle cx="12" cy="12" r="4"/><path d="M12 2v2m0 16v2M2
12h2m16 0h2M5 5l1.5 1.5m11 11L19 19M5 19l1.5-1.5m11-11L19 5"/></symbol>
23      <symbol id="icone-moto" viewBox="0 0 24 24"><circle cx="5" cy="17" r="3.5"/><circle cx="19" cy="17"
r="3.5"/><path d="M19 17 15 6h-3m-9 4h5l3 4h5M5 17l4-5m1 2 1 3h4m-7-7 4-2h4"/><path d="M8 10h4l3 3H10Z"
fill="currentColor" stroke="none"/></symbol>
24      <symbol id="icone-acessivel" viewBox="0 0 24 24"><circle cx="11" cy="4" r="1.5"/><path d="M11
7v6h6l3 7m-9-10h6M8 11a6 6 0 1 0 8 8"/></symbol>
```

```
  </svg>

  <!-- CABEÇALHO: logo, nome do sistema e botão que alterna o tema claro/escuro -->
  <header class="topo">
    <div class="miolo">
      <a class="marca" href="">
        <span class="logo"><img src="image/logo-provisoria.jpeg" alt="Logo"></span>
        <span>ParkingHub<small>Reserva de vagas</small></span>
      </a>
      <!-- Botão de tema: tem os dois ícones e o CSS mostra só um (sol no tema escuro, lua no claro) -->
      <button class="icone" id="tema" type="button" aria-label="Alternar tema">
        <svg class="icone-svg icone-sol"><use href="#icone-sol"/></svg><svg class="icone-svg icone-lua">
<use href="#icone-lua"/></svg>
      </button>
    </div>
  </header>
```

- O `<svg>` com `<filter>` é um truque visual: no tema claro, ele troca o preto da logo por branco para ela aparecer bem. Está invisível (largura 0). O CSS chama o filtro pelo id `logo-claro`.
- O segundo `<svg>` é a **biblioteca de ícones**: cada `<symbol>` (lua, sol, moto e acessível) é um desenho com um id. Em qualquer lugar, `<use href="#icone-moto">` reaproveita o desenho. Assim não precisamos de emojis nem de arquivos de imagem.
- `<header class="topo">` é o cabeçalho. Dentro dele, a logo, o nome "ParkingHub" e o botão `#tema`. Ele tem os dois ícones (sol e lua) e o CSS mostra só um, conforme o tema.
- `class` serve para o CSS estilizar; `id` é um nome único para o JavaScript achar o elemento.

Título e barra de etapas (linhas 41 a 46)

```
  <main class="miolo pagina">
    <h1>Reserve sua vaga</h1>
    <p class="sub">Escolha sua vaga, informe seus dados e siga para o pagamento.</p>

    <!-- Barra de progresso: o JavaScript marca qual etapa está ativa -->
    <ol class="etapas" id="etapas"><li>Vaga e período</li><li>Seus dados</li><li>Pagamento</li></ol>
```

- `<main>` é o conteúdo principal da página.
- `<ol id="etapas">` é uma lista numerada com as 3 etapas. O JS troca a classe de cada `<li>` para destacar a etapa atual (`atual`) e as já feitas (`feito`).

Etapa 1: período da reserva (linhas 48 a 66)

```
    <!-- ===== ETAPA 1: período, planta e resumo ===== -->
    <form class="reserva" id="tela-vaga" novalidate>

      <!-- Período: modalidade, data e horários -->
      <section class="periodo">
        <h2>Período da reserva</h2>
        <fieldset class="modo">
          <legend>Modalidade</legend>
          <!-- Os dois botões de rádio têm o mesmo "name", então só um pode ficar marcado -->
          <label><input type="radio" name="modalidade" value="horario" checked> Por horário</label>
          <label><input type="radio" name="modalidade" value="diaria"> Diária de 24 h</label>
        </fieldset>
        <label class="campo">Data de entrada <input type="date" id="data"></label>
        <div class="duas">
          <label class="campo">Entrada <input type="time" id="entrada" value="08:00"></label>
          <label class="campo" id="campo-saida">Saída <input type="time" id="saida" value="10:00">
```

```
</label> <!-- some na diária -->
 64          </div>
 65          <p class="erro" id="erro-periodo" hidden>Confira a data e os horários: a saída precisa ser
depois da entrada.</p> <!-- aparece quando o período é inválido -->
 66        </section>
```

- `<form id="tela-vaga">` agrupa toda a etapa 1. O JS captura o envio (clique em "Continuar") deste formulário.
- Dois `<input type="radio">` com o mesmo `name="modalidade"` formam um grupo em que só um pode ser marcado: "horario" (por horário) ou "diaria". O `value` é o que o JS lê.
- `type="date"` e `type="time"` mostram o seletor de data e de hora do próprio navegador. Os valores iniciais de horário (08:00 e 10:00) evitam que a tela comece sem período válido.
- `#campo-saida` é o campo de saída; o JS o esconde quando a modalidade é diária.
- `<p id="erro-periodo" hidden>` mostra mensagens de erro do período. O atributo `hidden` mantém o parágrafo oculto até ser necessário.

Etapa 1: planta do estacionamento (linhas 68 a 90)

```
 68        <!-- Planta: as fileiras estão VAZIAS; o JavaScript cria as 34 vagas dentro delas -->
 69        <section class="planta-box">
 70          <div class="titulo-planta">
 71            <div><h2>Escolha uma vaga</h2><p>Planta do estacionamento</p></div>
 72            <span id="disponiveis"></span> <!-- "34 de 34 disponíveis" -->
 73          </div>
 74          <div class="legenda">
 75            <span><i class="livre"></i>Disponível</span>
 76            <span><i class="selecionada"></i>Selecionada</span>
 77            <span><i class="reservada"></i>Reservada</span>
 78            <span><svg class="icone-svg" aria-hidden="true"><use href="#icone-acessivel"/></svg>
Acessível</span>
 79          </div>
 80          <div class="rolagem">
 81            <div class="planta">
 82              <div class="muro"></div>
 83              <div class="fileira topo-f" id="fileira-topo"></div>
 84              <div class="fileira meio-f" id="fileira-meio"></div>
 85              <div class="fileira base-f" id="fileira-base"></div>
 86              <div class="entrada"><span>←<small>Saída</small></span><span>→<small>Entrada</small></span>
</div>
 87            </div>
 88          </div>
 89          <p class="legenda-fim">V1–V26: carros · M1–M6: motos · D1–D2: acessíveis</p>
 90        </section>
```

- O título mostra "N de 34 disponíveis" em `#disponiveis` (preenchido pelo JS).
- A legenda usa tags `<i>` vazias, coloridas pelo CSS, como amostras de cor.
- `.rolagem` permite rolar a planta para o lado em telas pequenas.
- Dentro de `.planta` estão o muro desenhado e as **três fileiras vazias** (`#fileira-topo`, `#fileira-meio`, `#fileira-base`). Os botões das 34 vagas **não estão escritos aqui**: o JavaScript cria cada um dentro dessas divs.
- A div `.entrada` desenha as setas de entrada e saída.

Etapa 1: resumo e botão continuar (linhas 92 a 106)

```
92        <!-- Resumo: vaga, datas, duração e total (preenchidos pelo JavaScript) -->
93        <section class="resumo-box">
94          <h2>Confira sua reserva</h2>
95          <div class="vaga-escolhida"><span>Vaga selecionada</span><strong id="r-vaga"></strong><small
id="r-tipo"></small></div>
96          <dl>
97            <div><dt>Entrada</dt><dd id="r-entrada"></dd></div>
98            <div><dt>Saída</dt><dd id="r-saida"></dd></div>
99            <div><dt>Duração</dt><dd id="r-duracao"></dd></div>
100           <div class="total"><dt>Total da reserva</dt><dd id="r-total"></dd></div>
101         </dl>
102         <button class="principal" id="continuar" disabled>Selecione uma vaga</button> <!-- liberado só
com período e vaga válidos -->
103         <p class="aviso" id="aviso" role="status" hidden></p>                          <!-- mensagens de
sucesso ou de vaga ocupada -->
104         <p class="nota" id="tarifas"></p>
105       </section>
106     </form>
```

- Mostra a vaga selecionada, entrada, saída, duração e total. Cada valor está num `<dd>` com id (`r-vaga` , `r-total` ...) que o JS preenche.
- `<dl>` / `<dt>` / `<dd>` é a lista de definições: nome do dado e seu valor.
- O botão `#continuar` começa `disabled` (desabilitado) e só é liberado quando há período válido e vaga escolhida.
- `#aviso` mostra mensagens (sucesso da reserva, vaga que ficou ocupada) e `#tarifas` mostra os preços.

Etapas 2 e 3: tela de confirmação e resumo (linhas 108 a 124)

```
108     <!-- ===== ETAPAS 2 e 3: o resumo à esquerda é o mesmo; à direita aparece o formulário de dados OU o
de pagamento ===== -->
109     <section class="confirmar" id="tela-confirmar" hidden>
110       <button class="voltar" id="voltar" type="button">← Voltar</button>
111       <div class="duas-colunas">
112
113         <!-- Resumo lateral da reserva -->
114         <aside class="resumo-box">
115           <h2>Sua reserva</h2>
116           <div class="vaga-escolhida"><strong id="c-vaga"></strong><small id="c-tipo"></small></div>
117           <dl>
118             <div><dt>Modalidade</dt><dd id="c-mod"></dd></div>
119             <div><dt>Entrada</dt><dd id="c-entrada"></dd></div>
120             <div><dt>Saída</dt><dd id="c-saida"></dd></div>
121             <div id="c-cliente" hidden><dt>Cliente</dt><dd id="c-nome"></dd></div> <!-- só aparece na
etapa 3 -->
122             <div class="total"><dt>Total da reserva</dt><dd id="c-total"></dd></div>
123           </dl>
124         </aside>
```

- `#tela-confirmar` começa `hidden` . Ela é compartilhada pelas etapas 2 e 3: o resumo à esquerda é o mesmo e muda só o formulário à direita.
- `<aside>` é o resumo lateral (vaga, modalidade, entrada, saída, cliente, total). A linha do cliente (`#c-cliente`) só aparece na etapa 3.
- O botão "Voltar" tem `type="button"` para não enviar o formulário.

Etapa 2: formulário de dados (linhas 126 a 135)

```
126         <!-- ETAPA 2: dados do cliente e do veículo. "novalidate": a validação é feita pelo nosso
JavaScript -->
127         <form class="form-box" id="form-dados" novalidate>
128           <h2>Seus dados</h2>
129           <label class="campo">Nome completo <input id="nome" maxlength="120" autocomplete="name">
</label>
130           <label class="campo">Telefone com DDD (só números) <input id="telefone" type="tel"
maxlength="11" placeholder="11999998888"></label>
131           <h2>Dados do veículo</h2>
132           <label class="campo">Placa (7 caracteres) <input id="placa" maxlength="7"
placeholder="ABC1D23" autocomplete="off"></label>
133           <p class="erro" id="erro-dados" hidden></p>
134           <button class="principal">Ir para pagamento</button>
135         </form>
```

- Campos de nome, telefone e placa. `maxlength` limita a quantidade de caracteres, `placeholder` mostra o exemplo cinza dentro do campo e `type="tel"` abre o teclado numérico no celular.
- `#erro-dados` exibe a mensagem de erro quando algo está errado.
- `novalidate` desliga a validação automática do navegador, porque a conferência é feita pelo nosso JavaScript, com mensagens em português.

Etapa 3: formulário de pagamento (linhas 137 a 153)

```
137         <!-- ETAPA 3: forma de pagamento (o JavaScript lê qual opção está marcada) -->
138         <form class="form-box" id="form-pagamento" novalidate hidden>
139           <fieldset class="pagamentos">
140             <legend>Forma de pagamento</legend>
141             <label><input type="radio" name="pagamento" value="cartao"> <b>Cartão de crédito</b>
<small>Pagar no estacionamento</small></label>
142             <label><input type="radio" name="pagamento" value="google-wallet"> <b>Google Wallet</b>
<small>Carteira digital</small></label>
143             <label><input type="radio" name="pagamento" value="pix"> <b>Pix</b> <small>Simular com QR
Code</small></label>
144             <label><input type="radio" name="pagamento" value="dinheiro"> <b>Dinheiro</b> <small>Pagar
no estacionamento</small></label>
145           </fieldset>
146           <h2 id="pg-titulo">Escolha uma opção acima</h2>
147           <p id="pg-texto">O total é o mesmo em todas as formas de pagamento.</p>
148           <img id="pix" src="image/pix-demonstracao.svg" width="180" height="180" alt="QR Code
ilustrativo" hidden> <!-- só no Pix -->
149           <p class="nota">Ambiente de demonstração: nenhuma cobrança real é realizada.</p>
150           <button class="principal" id="pagar" disabled>Confirmar reserva</button>
151         </form>
152       </div>
153     </section>
```

- Quatro `radio` com `name="pagamento"`: cartão, Google Wallet, Pix e dinheiro. O `value` de cada um é a chave usada no objeto `textosPagamento` do JS.
- `<fieldset>` e `<legend>` agrupam e nomeiam as opções.
- O título e o texto (`#pg-titulo`, `#pg-texto`) mudam conforme a opção. A imagem do QR Code (`#pix`) só aparece no Pix.
- O botão `#pagar` começa desabilitado até escolher uma forma. Este formulário começa `hidden`.

Minhas reservas (linhas 155 a 162)

```
155     <!-- ===== MINHAS RESERVAS: a lista é desenhada pelo JavaScript dentro de #lista ===== -->
156     <section class="minhas" id="minhas">
157       <div class="titulo-minhas">
158         <h2>Minhas reservas <span id="contador">0</span></h2> <!-- quantidade de reservas ativas -->
159       </div>
160       <div id="lista"></div>
161     </section>
162   </main>
```

- Título com o contador de reservas ativas (`#contador`).
- `<div id="lista">` começa vazia. O JS desenha ali uma caixa para cada reserva, com o botão de cancelar.

Rodapé (linhas 164 a 178)

```
164   <!-- RODAPÉ: <details> abre e fecha ao clicar, sem precisar de JavaScript -->
165   <footer class="rodape miolo">
166     <span>Projeto de gerenciamento de estacionamento</span>
167     <details><summary>Equipe do projeto</summary>
168       <ul>
169         <li>Paulo Henrique Alves de Almeida</li>
170         <li>Matheus Avanzo dos Santos</li>
171         <li>Murilo Roselini</li>
172         <li>Petterson Augusto Papa de Souza</li>
173         <li>Raphael Souza Araujo Barbosa</li>
174       </ul>
175     </details>
176   </footer>
177 </body>
178 </html>
```

- `<footer>` mostra o nome do projeto e a equipe dentro de `<details>`, um bloco que abre e fecha ao clicar em `<summary>`, sem precisar de JavaScript.

Parte 2 - style.css

Fontes (linhas 1 a 6)

```
1  /* ParkingHub - aparência do sistema (versão simples).
2     O CSS só cuida do visual: o JavaScript liga e desliga classes e atributos, e as regras abaixo reagem
a eles. */
3
4  /* Fontes locais (pasta fonts): Source Sans 3 para o texto e Barlow para os títulos. */
5  @font-face { font-family: "Source Sans 3"; src: url("fonts/SourceSans3-Variable.ttf"); font-weight: 200
900; }
6  @font-face { font-family: "Barlow"; src: url("fonts/Barlow-SemiBold.ttf"); font-weight: 600; }
```

- `@font-face` carrega as fontes da pasta `fonts`: Source Sans 3 (texto) e Barlow (títulos). As fontes ficam na pasta `fonts`, na raiz do projeto.

Variáveis de cor e tema escuro (linhas 8 a 39)

```
8  /* Cores do tema claro, guardadas em variáveis (usadas com var(--nome)).
9     Mudar uma cor aqui muda o site inteiro. */
10 :root {
11   --fundo: #f5f6f7;            /* fundo da página */
12   --superficie: #fff;          /* fundo de cartões, campos e botões */
13   --suave: #e9ecef;            /* cinza claro de apoio */
14   --texto: #191d21;            /* cor do texto principal */
15   --secundario: #505c66;       /* cor de textos de apoio */
16   --borda: #cdd3d8;            /* cor das bordas */
17   --destaque: #c44100;         /* laranja da marca */
18   --destaque-suave: #fff1e7;   /* laranja bem claro (fundos de destaque) */
19   --sobre-destaque: #fff;      /* cor do texto sobre o laranja */
20   --piso: #e9ecef;             /* piso do estacionamento */
21   --marcacao: #77838e;         /* linhas desenhadas na planta */
22   --reservada: #d6dadd;        /* cinza das vagas reservadas */
23   --azul: #24669a;             /* azul das vagas acessíveis */
24   --vermelho: #ab3522;         /* mensagens de erro */
25   --cabecalho: #e9eaec;        /* fundo do cabeçalho */
26   --cabecalho-texto: #17191c;  /* texto do cabeçalho */
27   --fonte-texto: "Source Sans 3", "Segoe UI", sans-serif;
28   --fonte-titulo: "Barlow", "Segoe UI", sans-serif;
29 }
30
31 /* Tema escuro: só REDEFINE as mesmas variáveis. Vale quando o elemento raiz tem data-tema="escuro"
32    (o JavaScript escreve esse atributo no botão de tema). */
33 :root[data-tema="escuro"] {
34   color-scheme: dark;
35   --fundo: #101214; --superficie: #16191c; --suave: #242a30; --texto: #f0f1f2; --secundario: #b0bac3;
36   --borda: #343b42; --destaque: #ff7b29; --destaque-suave: #35251b; --sobre-destaque: #101214;
37   --piso: #0c0f11; --reservada: #323940; --azul: #88bce8; --vermelho: #ffb0a0;
38   --cabecalho: #000; --cabecalho-texto: #fff;
39 }
```

- `:root` é o elemento raiz da página. As variáveis `--nome` guardam as cores e são usadas com `var(--nome)`. Mudar uma cor aqui muda o site todo.
- `:root[data-tema="escuro"]` só vale quando o elemento raiz tem `data-tema="escuro"`. Ele **redefine as mesmas variáveis** com cores escuras. É assim que o tema escuro funciona: o JS troca o atributo e o CSS faz o resto.
- `--destaque` é o laranja da marca. `--fonte-texto` e `--fonte-titulo` são as fontes.

Estilos base (linhas 41 a 53)

```
41  /* ---------- Estilos base ---------- */
42  * { box-sizing: border-box; }  /* largura e altura incluem borda e espaçamento interno */
43  body { margin: 0; background: var(--fundo); color: var(--texto); font: 16px/1.5 var(--fonte-texto); }
44  h1, h2, p, dl { margin: 0; }
45  h1, h2 { font-family: var(--fonte-titulo); font-weight: 600; }
46  h1 { font-size: 36px; line-height: 1.15; }
47  h2 { font-size: 22px; }
48  button, input, select { font: inherit; }  /* campos usam a mesma fonte da página */
49  button { cursor: pointer; }
50  [hidden] { display: none !important; }    /* qualquer elemento com o atributo "hidden" some de verdade
*/
51  .miolo { width: min(1440px, calc(100% - 64px)); margin: 0 auto; }  /* limita a largura e centraliza o
conteúdo */
52  .pagina { padding: 28px 0 32px; }
53  .sub { color: var(--secundario); font-size: 15px; margin: 4px 0 20px; }
```

- `box-sizing: border-box`: largura e altura incluem borda e espaçamento interno (fica mais fácil calcular tamanhos).
- `body` define cor de fundo, cor do texto e fonte padrão. `[hidden] { display: none !important }` garante que qualquer elemento com o atributo `hidden` realmente suma, mesmo que outra regra tente exibi-lo.
- `.miolo` limita a largura do conteúdo (máximo de 1440 px) e o centraliza com `margin: 0 auto`.

Cabeçalho e logo (linhas 55 a 70)

```
55  /* ---------- Cabeçalho ---------- */
56  .topo { background: var(--cabecalho); color: var(--cabecalho-texto); border-bottom: 1px solid var(--
borda); padding: 8px 0; }
57  .topo .miolo { display: flex; justify-content: space-between; align-items: center; }  /* marca à
esquerda, tema à direita */
58  .marca { display: flex; align-items: center; gap: 12px; color: inherit; text-decoration: none; font: 600
21px/1.3 var(--fonte-titulo); }
59  .marca small { display: block; font: 12px var(--fonte-texto); opacity: .7; }
60  /* A logo é uma imagem maior dentro de uma caixa de 60 px que recorta só o símbolo. */
61  .logo { width: 60px; height: 60px; overflow: hidden; position: relative; }
62  .logo img { position: absolute; width: 112px; left: -26px; top: -24px; filter: url("#logo-claro"); }  /*
filtro do SVG no HTML (tema claro) */
63  :root[data-tema="escuro"] .logo img { filter: none; }
64  .icone { width: 44px; height: 44px; border: 0; background: none; color: inherit; display: inline-flex;
align-items: center; justify-content: center; }
65  /* Ícones SVG: desenhados só com traço, que usa a cor do texto (currentColor). Os desenhos ficam no
index.html. */
66  .icone-svg { width: 19px; height: 19px; flex: 0 0 19px; fill: none; stroke: currentColor; stroke-width:
1.7; stroke-linecap: round; stroke-linejoin: round; }
67  .icone .icone-svg { width: 24px; height: 24px; }                                        /*
ícone do tema, no cabeçalho */
68  :root[data-tema="claro"] .icone-sol, :root[data-tema="escuro"] .icone-lua { display: none; }      /*
mostra só um dos dois ícones do botão de tema */
69  .vaga.acessivel .icone-svg { width: 25px; height: 25px; flex-basis: 25px; }                      /*
vaga acessível: ícone maior, na cor azul da vaga */
70  .legenda .icone-svg { width: 17px; height: 17px; flex-basis: 17px; color: var(--azul); vertical-align:
-4px; }  /* legenda */
```

- `display: flex` com `justify-content: space-between` coloca a marca à esquerda e o botão de tema à direita.
- A logo é uma imagem maior dentro de uma caixa de 60 px com `overflow: hidden`: a caixa recorta só o símbolo. `filter: url("#logo-claro")` aplica o filtro do SVG (só no tema claro).

- `.icone-svg` define o tamanho dos ícones e usa `stroke: currentColor`: o traço do desenho assume a cor do texto ao redor (branco, azul na vaga acessível, etc.). `fill: none` deixa o desenho só com contorno.

Barra de etapas (linhas 72 a 77)

```
 72  /* ---------- Barra de etapas (os números 1., 2., 3. são gerados automaticamente) ---------- */
 73  .etapas { display: flex; gap: 8px; padding: 0; margin: 0 0 24px; list-style: none; counter-reset: etapa;
}
 74  .etapas li { counter-increment: etapa; flex: 1; padding: 8px 0; border-top: 3px solid var(--borda);
color: var(--secundario); font-size: 14px; }
 75  .etapas li::before { content: counter(etapa) ". "; }
 76  .etapas .atual { border-color: var(--destaque); color: var(--texto); font-weight: 600; }  /* etapa em
andamento */
 77  .etapas .feito { border-color: var(--texto); }                                            /* etapas já
concluídas */
```

- Cada `<li>` vira uma faixa com borda superior. `counter-reset` e `counter-increment` numeram automaticamente (1., 2., 3.) pelo `::before`.
- `.atual` (etapa em andamento) fica com borda laranja e negrito; `.feito` fica com borda escura. As classes são colocadas pelo JS.

Campos, botões e mensagens (linhas 79 a 94)

```
 79  /* ---------- Campos, botões e mensagens ---------- */
 80  .campo { display: flex; flex-direction: column; gap: 4px; margin-top: 14px; font-size: 14px; }  /*
rótulo em cima, campo embaixo */
 81  input, select { padding: 9px 10px; border: 1px solid var(--secundario); border-radius: 3px; background:
var(--superficie); color: inherit; }
 82  fieldset { border: 0; padding: 0; margin: 14px 0 0; }
 83  legend { font-size: 14px; padding: 0; margin-bottom: 4px; }
 84  .modo label { margin-right: 16px; }
 85  .duas { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }  /* dois campos lado a lado */
 86  .principal { width: 100%; margin-top: 16px; padding: 12px; border: 0; border-radius: 3px; background:
var(--destaque); color: var(--sobre-destaque); font-weight: 600; }
 87  .principal:hover:enabled { filter: brightness(1.1); }                              /* clareia ao passar
o mouse */
 88  .principal:disabled { background: var(--suave); color: var(--secundario); cursor: not-allowed; }  /*
botão desabilitado fica cinza */
 89  .botao { padding: 8px 14px; border: 1px solid var(--borda); border-radius: 3px; background: var(--
superficie); color: inherit; }
 90  .botao:hover { background: var(--suave); }
 91  .voltar { border: 0; background: none; color: var(--texto); padding: 8px 0; margin-bottom: 12px; }
 92  .erro { color: var(--vermelho); font-size: 14px; margin-top: 10px; }
 93  .aviso { margin-top: 12px; padding: 10px; border-left: 4px solid var(--destaque); background: var(--
destaque-suave); font-size: 14px; }
 94  .nota { color: var(--secundario); font-size: 13px; margin-top: 8px; }
```

- `.campo`: rótulo e input empilhados (`flex-direction: column`) com espaço entre eles.
- `.principal`: botão laranja de largura total. `:hover` clareia ao passar o mouse e `:disabled` deixa cinza quando está desabilitado.
- `.botao` é o botão secundário (borda cinza). `.erro` é texto vermelho e `.aviso` é a caixa de mensagem com barra laranja à esquerda.

Layout da etapa 1 (linhas 96 a 109)

```
 96  /* ---------- Etapa 1: planta à esquerda, período e resumo à direita (CSS Grid) ---------- */
 97  .reserva { display: grid; grid-template-columns: minmax(0, 1fr) 344px; grid-template-areas: "planta
periodo" "planta resumo"; align-items: start; column-gap: 32px; }
 98  .planta-box { grid-area: planta; background: var(--superficie); border: 1px solid var(--borda);
overflow: hidden; }
 99  .periodo { grid-area: periodo; padding-bottom: 24px; }
100  .resumo-box { grid-area: resumo; padding-top: 24px; border-top: 1px solid var(--borda); }
101  .titulo-planta { display: flex; justify-content: space-between; align-items: center; padding: 22px 24px
12px; }
102  .titulo-planta p, .titulo-planta span { color: var(--secundario); font-size: 14px; }
103  /* Legenda: quadradinhos coloridos que explicam as cores das vagas. */
104  .legenda { display: flex; flex-wrap: wrap; gap: 18px; padding: 0 24px 16px; color: var(--secundario);
font-size: 13px; }
105  .legenda i { display: inline-block; width: 14px; height: 14px; margin-right: 6px; vertical-align: -2px;
border: 1px solid var(--marcacao); }
106  .legenda .livre { background: var(--superficie); }
107  .legenda .selecionada { background: var(--destaque); }
108  .legenda .reservada { background: var(--reservada); }
109  .legenda-fim { padding: 14px 24px; color: var(--secundario); font-size: 12px; }
```

- `.reserva` usa **CSS Grid**: duas colunas (a primeira ocupa o espaço restante, a segunda tem 344 px). `grid-template-areas` posiciona: a planta ocupa as duas linhas da coluna da esquerda, e período e resumo ficam empilhados na direita.
- `grid-area` em cada bloco diz em qual área ele fica.
- A legenda usa quadradinhos coloridos (`.legenda i`) para mostrar disponível, selecionada e reservada.

A planta e as vagas (linhas 111 a 130)

```
111  /* ---------- Planta: posições em porcentagem, para o desenho acompanhar o tamanho da tela ---------- */
112  .rolagem { overflow-x: auto; border-block: 1px solid var(--borda); }  /* em telas pequenas, rola para o
lado */
113  .planta { position: relative; min-width: 950px; aspect-ratio: 1.286; background: var(--piso); }
114  .muro { position: absolute; inset: 3%; border: 2px solid var(--marcacao); }  /* contorno do
estacionamento */
115  .fileira { position: absolute; display: grid; }  /* cada fileira é uma grade de vagas */
116  .topo-f { left: 19%; top: 7%; width: 69%; height: 18%; grid-template-columns: repeat(7, 1fr); }   /* 7
colunas: motos + V1 a V6 */
117  .meio-f { left: 16%; top: 33%; width: 78%; height: 36%; grid-template-columns: repeat(8, 1fr); grid-
template-rows: 1fr 1fr; }  /* 8 colunas x 2 linhas: 16 vagas */
118  .base-f { left: 19%; top: 77%; width: 69%; height: 18%; grid-template-columns: repeat(7, 1fr); }  /* 7
colunas: motos + V21 a V26 */
119  .motos { display: grid; grid-template-rows: repeat(3, 1fr); }  /* pilha de 3 vagas de moto */
120
121  /* Cada vaga é um botão. */
122  .vaga { display: flex; flex-direction: column; align-items: center; justify-content: center; gap: 4px;
min-width: 0; padding: 4px; border: 1px solid var(--marcacao); background: var(--superficie); color: var(--
texto); font-weight: 600; }
123  .vaga.moto { flex-direction: row; }
124  .vaga.acessivel { color: var(--azul); }
125  .vaga:enabled:hover { background: var(--destaque-suave); outline: 2px solid var(--destaque); outline-
offset: -2px; }  /* destaque ao passar o mouse */
126  .vaga.selecionada { background: var(--destaque); color: var(--sobre-destaque); }
/* vaga escolhida: laranja */
127  .vaga:disabled { background: var(--reservada); color: var(--secundario); cursor: not-allowed; }
/* reservada ou sem período válido: cinza */
128  /* Setas de entrada e saída desenhadas na lateral da planta. */
129  .entrada { position: absolute; left: 2%; top: 35%; width: 9%; height: 32%; border-right: 2px dashed
var(--marcacao); display: flex; flex-direction: column; justify-content: space-around; text-align: center;
```

```
color: var(--secundario); font: 32px/1 var(--fonte-titulo); }
130  .entrada small { display: block; font: 11px var(--fonte-texto); }
```

- `.planta` tem `position: relative`, e as fileiras `position: absolute` com `left`, `top`, `width` e `height` em **porcentagem**. Assim o desenho mantém a proporção em qualquer largura. `aspect-ratio` mantém o formato; `min-width: 950px` impede que fique pequena demais (aí a rolagem lateral entra).
- Cada fileira é um **Grid**: `repeat(7, 1fr)` cria 7 colunas iguais. A do meio tem 8 colunas e 2 linhas (16 vagas).
- `.motos` é uma pilha de 3 linhas para as vagas de moto.
- `.vaga` é cada vaga: botão com borda, texto centralizado. `.vaga:enabled:hover` destaca ao passar o mouse; `.vaga.selecionada` pinta de laranja; `.vaga:disabled` deixa cinza (reservada ou sem período válido).

Resumo da reserva (linhas 132 a 140)

```
132  /* ---------- Resumo da reserva ---------- */
133  .vaga-escolhida { margin: 12px 0; padding: 12px; background: var(--superficie); border: 1px solid var(--
borda); }  /* caixa com o código da vaga */
134  .vaga-escolhida span, .vaga-escolhida small { display: block; color: var(--secundario); font-size: 13px;
}
135  .vaga-escolhida strong { font: 600 28px var(--fonte-titulo); }
136  /* Cada par "nome / valor" ocupa uma linha, com o nome à esquerda e o valor à direita. */
137  dl div { display: flex; justify-content: space-between; gap: 12px; padding: 8px 0; border-bottom: 1px
solid var(--borda); font-size: 14px; }
138  dt { color: var(--secundario); }
139  dd { margin: 0; font-weight: 600; text-align: right; }
140  .total dd { font-size: 20px; color: var(--destaque); }
```

- `.vaga-escolhida` é a caixa grande com o código da vaga.
- `dl div` faz cada par "nome / valor" ficar numa linha, com o nome à esquerda e o valor à direita (`justify-content: space-between`). `.total dd` destaca o total em laranja.

Layout das etapas 2 e 3 e pagamento (linhas 142 a 151)

```
142  /* ---------- Etapas 2 e 3: resumo à esquerda e formulário à direita ---------- */
143  .duas-colunas { display: grid; grid-template-columns: 344px minmax(0, 560px); gap: 32px; align-items:
start; }
144  .duas-colunas .resumo-box { padding-top: 0; border: 0; }
145  .form-box { background: var(--superficie); border: 1px solid var(--borda); padding: 24px; }
146  .form-box h2 { margin-top: 20px; }
147  .form-box h2:first-child { margin-top: 0; }
148  .pagamentos label { display: block; padding: 12px; margin-bottom: 8px; border: 1px solid var(--borda); }
149  .pagamentos label:has(:checked) { border-color: var(--destaque); background: var(--destaque-suave); }
/* :has() destaca a opção marcada, sem JavaScript */
150  .pagamentos small { display: block; margin-left: 24px; color: var(--secundario); }
151  #pix { display: block; margin-top: 12px; }  /* QR Code de demonstração */
```

- `.duas-colunas`: resumo de 344 px e formulário de até 560 px.
- `.pagamentos label:has(:checked)` usa o seletor `:has()` para destacar com borda laranja a opção de pagamento que está marcada, sem JavaScript.

Lista de reservas (linhas 153 a 160)

```
153  /* ---------- Minhas reservas ---------- */
154  .minhas { margin-top: 40px; }
155  .titulo-minhas { display: flex; justify-content: space-between; align-items: center; margin-bottom:
```

```
12px; }
156  #contador { background: var(--superficie); border: 1px solid var(--borda); padding: 2px 8px; font: 14px
var(--fonte-texto); }
157  /* Cada reserva é uma caixa com o código da vaga, os detalhes e o botão de cancelar. */
158  .recibo { display: flex; align-items: center; flex-wrap: wrap; gap: 16px; padding: 16px; margin-bottom:
8px; background: var(--superficie); border: 1px solid var(--borda); }
159  .recibo p { flex: 1; min-width: 220px; }
160  .recibo strong.codigo { font: 600 28px var(--fonte-titulo); color: var(--destaque); min-width: 56px; }
/* código da vaga em laranja */
```

- Cada reserva (`.recibo`) é uma caixa flex: o código da vaga em laranja e grande, os dados ao lado e os botões à direita. `flex-wrap: wrap` permite quebrar de linha em telas pequenas.

Rodapé (linhas 162 a 163)

```
162  /* ---------- Rodapé ---------- */
163  .rodape { padding: 20px 0 32px; border-top: 1px solid var(--borda); color: var(--secundario); font-size:
14px; }
```

- Linha separadora em cima, texto menor e em cor de apoio.

Responsividade (linhas 165 a 171)

```
165  /* ---------- Telas menores (até 1000 px): tudo em uma coluna ---------- */
166  @media (max-width: 1000px) {
167    .miolo { width: calc(100% - 32px); }
168    .reserva { grid-template-columns: 1fr; grid-template-areas: "periodo" "planta" "resumo"; }  /* ordem:
período, planta, resumo */
169    .duas-colunas { grid-template-columns: 1fr; }
170    .etapas li { font-size: 12px; }
171  }
```

- `@media (max-width: 1000px)` vale só em telas de até 1000 px de largura. Nela, a grade vira **uma coluna** e a ordem passa a ser período, planta e resumo. As etapas 2 e 3 também empilham.

Parte 3 - script.js

> O arquivo segue a jornada do usuário: variáveis, planta, período e regras, etapa 1, etapa 2, etapa 3, minhas reservas e tema.

Variáveis e tarifas (linhas 1 a 11)

```
1  // ParkingHub - reserva de vagas de estacionamento (versão simples)
2
3  // ----- 1) Variáveis -----
4  const VALOR_HORA = 8;        // tarifas de exemplo, em reais
5  const VALOR_DIARIA = 35;
6  const MS_POR_HORA = 3600000; // o JavaScript guarda datas em milissegundos: 1 hora = 3.600.000 ms
7
8  let vagas = [];            // as 34 vagas: cada uma é { codigo, tipo, botao }
9  let reservas = [];         // todas as reservas feitas
10 let vagaEscolhida = null;  // vaga clicada na planta (null = nenhuma)
11 let etapaAtual = 1;        // 1 = vaga e período, 2 = seus dados, 3 = pagamento
```

- `const` cria uma variável que **não pode ser trocada** depois. `let` cria uma que pode mudar.
- `VALOR_HORA` e `VALOR_DIARIA` são as tarifas. Para mudar os preços, basta alterar esses dois números.
- `MS_POR_HORA = 3600000`: o JavaScript guarda datas em **milissegundos**. Uma hora tem 60 x 60 x 1000 = 3.600.000 ms. Isso facilita somar e comparar horários.
- `vagas` e `reservas` são **listas** (arrays). `vagaEscolhida` guarda a vaga clicada (ou `null`, que significa "nada"). `etapaAtual` guarda em qual das 3 etapas estamos.

Salvar e carregar: localStorage (linhas 13 a 19)

```
13 // Carrega as reservas salvas no navegador. O localStorage guarda só texto, então usamos JSON.parse.
14 if (localStorage.getItem("reservas-parkinghub") != null) {
15   reservas = JSON.parse(localStorage.getItem("reservas-parkinghub"));
16 }
17 function salvarReservas() {
18   localStorage.setItem("reservas-parkinghub", JSON.stringify(reservas)); // lista -> texto
19 }
```

- O `localStorage` é uma "gaveta" do navegador que guarda **só texto** e não some ao recarregar a página.
- Ao abrir a página, se existe algo salvo (`!= null`), `JSON.parse` transforma o texto de volta em lista de reservas.
- `salvarReservas()` faz o caminho contrário: `JSON.stringify` transforma a lista em texto e `setItem` guarda com o nome "reservas-parkinghub".

Funções auxiliares (linhas 21 a 25)

```
21 function pegar(id) { return document.getElementById(id); } // atalho: pegar("nome") acha o elemento com id="nome"
22 function moeda(valor) { return "R$ " + valor.toFixed(2).replace(".", ","); }  // 16 -> "R$ 16,00"
23 function dataHora(ms) { return new Date(ms).toLocaleString("pt-BR", { dateStyle: "short", timeStyle: "short" }); }
24 function ehDiaria() { return document.querySelector('input[name="modalidade"]:checked').value == "diaria"; }
```

```
25 function icone(nome) { return '<svg class="icone-svg"><use href="#icone-' + nome + '"/></svg>'; } //
desenhos do index.html
```

- `pegar("nome")` é um atalho para `document.getElementById("nome")`: devolve o elemento da página que tem aquele `id`. É o que liga o JavaScript ao HTML.
- `moeda(16)` devolve "R$ 16,00": `toFixed(2)` deixa 2 casas decimais e `replace` troca o ponto pela vírgula.
- `dataHora` recebe milissegundos e devolve uma data legível, como "26/10/2026, 08:00".
- `ehDiaria()` olha qual botão de modalidade está marcado e devolve `true` ou `false`.
- `icone(nome)` devolve um ícone desenhado no `index.html` (moto ou cadeirante).

criarVaga(): cria o botão de uma vaga (linhas 27 a 43)

```
27 // ----- 2) Planta: 26 vagas de carro (V), 6 de moto (M) e 2 acessíveis (D) -----
28 function criarVaga(codigo, tipo, fileira, classe) {
29   const botao = document.createElement("button");
30   botao.type = "button";
31   botao.className = "vaga " + classe;
32   botao.innerHTML = codigo;
33   if (tipo == "Moto") botao.innerHTML += icone("moto");
34   if (tipo == "Acessível") botao.innerHTML += icone("acessivel");
35
36   const vaga = { codigo: codigo, tipo: tipo, botao: botao };
37   botao.onclick = function () {  // ao clicar: escolhe a vaga (ou desmarca, se já era a escolhida)
38     vagaEscolhida = (vagaEscolhida == vaga) ? null : vaga;
39     atualizar();
40   };
41   vagas.push(vaga);
42   fileira.append(botao);
43 }
```

- As 34 vagas **não estão escritas no HTML**: o JavaScript as cria. `document.createElement("button")` cria um botão novo, `className` define o estilo (CSS) e `innerHTML` o conteúdo: o código da vaga e, se for moto ou acessível, o ícone.
- O objeto `vaga = { codigo, tipo, botao }` junta as informações da vaga. Ele é guardado na lista `vagas` com `push`, e o botão é colocado na fileira com `fileira.append`.
- `botao.onclick = function () {...}` define o que acontece ao clicar: se a vaga já era a escolhida, desmarca; senão, escolhe. O `(condição) ? a : b` é um "se curto". Depois chama `atualizar()` para redesenhar a tela.

Repetindo vagas com laços (linhas 45 a 53)

```
45 function criarMotos(fileira, primeira) {  // pilha de 3 vagas de moto
46   const pilha = document.createElement("div");
47   pilha.className = "motos";
48   fileira.append(pilha);
49   for (let n = primeira; n < primeira + 3; n++) criarVaga("M" + n, "Moto", pilha, "moto");
50 }
51 function criarCarros(fileira, de, ate) {
52   for (let n = de; n <= ate; n++) criarVaga("V" + n, "Carro", fileira, "");
53 }
```

- `criarMotos` cria uma pilha de 3 vagas de moto. `for (let n = primeira; n < primeira + 3; n++)` é um **laço** (repetição) com três partes: onde começa, até quando continua e o que soma a cada volta.
- `"M" + n` junta texto e número: com n = 1 vira "M1".

- `criarCarros` repete de um número até outro para criar V1, V2, V3...

Montando a planta (linhas 55 a 62)

```
55  criarMotos(pegar("fileira-topo"), 1);       // fileira de cima: M1 a M3 e V1 a V6
56  criarCarros(pegar("fileira-topo"), 1, 6);
57  criarVaga("D1", "Acessível", pegar("fileira-meio"), "acessivel");  // fileira do meio: D1, V7 a V13, D2,
V14 a V20
58  criarCarros(pegar("fileira-meio"), 7, 13);
59  criarVaga("D2", "Acessível", pegar("fileira-meio"), "acessivel");
60  criarCarros(pegar("fileira-meio"), 14, 20);
61  criarMotos(pegar("fileira-base"), 4);       // fileira de baixo: M4 a M6 e V21 a V26
62  criarCarros(pegar("fileira-base"), 21, 26);
```

- Cada linha coloca vagas em uma das três fileiras do HTML, na ordem do desenho.
- Cima: M1 a M3 e V1 a V6. Meio: D1, V7 a V13, D2, V14 a V20. Baixo: M4 a M6 e V21 a V26.
- Total: 26 de carro + 6 de moto + 2 acessíveis = **34 vagas**.

calcularPeriodo(): início, fim e valor (linhas 64 a 76)

```
64  // ----- 3) Período, valor e conflitos -----
65
66  // Lê a data e os horários e devolve { inicio, fim, valor }. Devolve null se estiver inválido.
67  function calcularPeriodo() {
68    const dia = pegar("data").value;
69    const inicio = new Date(dia + "T" + pegar("entrada").value).getTime();
70    const fim = ehDiaria() ? inicio + 24 * MS_POR_HORA : new Date(dia + "T" +
pegar("saida").value).getTime();
71    if (isNaN(inicio) || isNaN(fim) || fim <= inicio) return null;  // isNaN = "não é número" (data vazia)
72
73    let valor = VALOR_DIARIA;
74    if (!ehDiaria()) valor = Math.ceil((fim - inicio) / MS_POR_HORA) * VALOR_HORA;  // ceil arredonda p/
cima: 2h30 = 3 horas
75    return { inicio: inicio, fim: fim, valor: valor };
76  }
```

- Lê a data e os horários do formulário. `pegar("data").value` é um texto como "2026-10-26". Juntando `dia + "T" + hora` fica "2026-10-26T08:00", que o `new Date(...)` entende. O `.getTime()` converte para milissegundos.
- Se a modalidade é diária, o fim é o início + 24 horas. Senão, o fim é a data com o horário de saída.
- `isNaN(inicio)` pergunta "não é um número?". Acontece quando a data está vazia. Se alguma data é inválida ou o fim não é depois do início, devolve `null` (período inválido).
- **Valor:** a diária é fixa. Por horário, `Math.ceil` arredonda as horas **para cima** (2h30 vira 3 horas) e multiplica pela tarifa: 3 x 8 = R$ 24.
- Devolve um objeto `{ inicio, fim, valor }`.

temConflito(): a regra mais importante (linhas 78 a 87)

```
78  // Já existe reserva ativa com essa vaga (campo = "vaga") ou placa (campo = "placa") nesse período?
79  // Dois períodos se sobrepõem quando cada um começa antes de o outro terminar.
80  function temConflito(campo, valor, periodo) {
81    for (const reserva of reservas) {
82      if (reserva.status == "ativa" && reserva[campo] == valor) {
83        if (periodo.inicio < reserva.fim && periodo.fim > reserva.inicio) return true;
```

```
    }
  }
  return false;
}
```

- Parâmetros: `campo` ("vaga" ou "placa"), `valor` (qual vaga ou placa) e `periodo` (o período desejado).
- `for (const reserva of reservas)` passa por cada reserva da lista.
- Para cada uma, o primeiro `if` vê se está **ativa** e se é da mesma vaga/placa (`reserva[campo]` lê a propriedade cujo nome está em `campo`).
- O segundo `if` vê se os horários **se sobrepõem**: `periodo.inicio < reserva.fim && periodo.fim > reserva.inicio`, ou seja, cada um começa antes de o outro terminar. Se sim, devolve `true` (tem conflito). Se terminar a lista sem achar, devolve `false`.

Reserva existente	Nova	Conflito?
08h às 10h	09h às 11h	Sim (09 < 10 e 11 > 08)
08h às 10h	10h às 12h	Não (10 não é menor que 10)
08h às 10h	06h às 07h	Não (07 não é maior que 08)

A mesma função serve para a vaga (bloqueia na planta) e para a placa (etapa 2). Reservas **canceladas** são ignoradas, por isso a vaga volta a ficar livre.

atualizar(): redesenha a etapa 1 (linhas 89 a 125)

```
// ----- 4) Etapa 1: vaga e período -----

// Redesenha a etapa 1. Roda toda vez que algo muda (data, horário, vaga clicada).
function atualizar() {
  const periodo = calcularPeriodo();
  pegar("campo-saida").hidden = ehDiaria();        // na diária não há horário de saída
  pegar("erro-periodo").hidden = periodo != null;  // aviso só se o período for inválido

  // Se a vaga escolhida ficou ocupada no novo período, desmarca.
  if (vagaEscolhida != null && periodo != null && temConflito("vaga", vagaEscolhida.codigo, periodo)) {
    vagaEscolhida = null;
    pegar("aviso").textContent = "A vaga escolhida está ocupada nesse período. Escolha outra.";
    pegar("aviso").hidden = false;
  }

  // Passa por todas as vagas: bloqueia as ocupadas, destaca a escolhida e conta as livres.
  let livres = 0;
  for (const vaga of vagas) {
    const ocupada = periodo != null && temConflito("vaga", vaga.codigo, periodo);
    vaga.botao.disabled = periodo == null || ocupada;
    vaga.botao.classList.toggle("selecionada", vaga == vagaEscolhida);
    if (periodo != null && !ocupada) livres++;
  }

  // Resumo ("—" quando o período é inválido)
  pegar("disponiveis").textContent = periodo ? livres + " de " + vagas.length + " disponíveis" : "Defina o período";
  pegar("r-entrada").textContent = periodo ? dataHora(periodo.inicio) : "—";
  pegar("r-saida").textContent = periodo ? dataHora(periodo.fim) : "—";
  pegar("r-duracao").textContent = periodo ? Math.round((periodo.fim - periodo.inicio) / MS_POR_HORA) + " hora(s)" : "—";
  pegar("r-total").textContent = periodo ? moeda(periodo.valor) : "—";
  pegar("r-vaga").textContent = vagaEscolhida ? vagaEscolhida.codigo : "Escolha na planta";
```

```
120    pegar("r-tipo").textContent = vagaEscolhida ? vagaEscolhida.tipo : "Carro, moto ou vaga acessível";
121
122    // O botão só é liberado com período válido e vaga escolhida.
123    pegar("continuar").disabled = periodo == null || vagaEscolhida == null;
124    pegar("continuar").textContent = vagaEscolhida ? "Continuar com a vaga " + vagaEscolhida.codigo :
"Selecione uma vaga";
125  }
```

- Roda **toda vez que algo muda** (data, horário ou vaga clicada). Recalcula o período e atualiza a tela.
- `pegar("campo-saida").hidden = ehDiaria()` esconde o campo de saída na diária. `hidden` é um atributo que esconde o elemento.
- Se a vaga escolhida ficou ocupada no novo período, mostra um aviso e desfaz a escolha.
- O laço `for (const vaga of vagas)` passa pelas 34 vagas: calcula se está `ocupada`, bloqueia o botão (`disabled`) se estiver ocupada ou o período for inválido, e liga/desliga a classe `selecionada` (laranja) com `classList.toggle`. Também conta as livres.
- O resumo usa `periodo ? A : B`: se existe período, mostra A; senão, mostra B (um traço). A duração é a diferença entre fim e início, dividida por `MS_POR_HORA`.
- O botão "Continuar" só é liberado com período válido **e** vaga escolhida, e o texto dele muda conforme a situação.

mostrarEtapa(): troca de etapa (linhas 127 a 141)

```
127  // Mostra a etapa 1, 2 ou 3 e esconde as outras (o atributo "hidden" esconde o elemento).
128  function mostrarEtapa(numero) {
129    etapaAtual = numero;
130    pegar("tela-vaga").hidden = numero != 1;
131    pegar("tela-confirmar").hidden = numero == 1;
132    pegar("form-dados").hidden = numero != 2;
133    pegar("form-pagamento").hidden = numero != 3;
134    pegar("minhas").hidden = numero != 1;
135    pegar("c-cliente").hidden = numero != 3;
136    const itens = pegar("etapas").children;  // barra de progresso: "atual" na etapa em andamento, "feito"
nas anteriores
137    for (let i = 0; i < itens.length; i++) {
138      itens[i].className = (i + 1 == numero) ? "atual" : (i + 1 < numero) ? "feito" : "";
139    }
140    window.scrollTo(0, 0);
141  }
```

- Recebe o número da etapa (1, 2 ou 3) e mostra/esconde as telas com `hidden`. Ex.: `pegar("tela-vaga").hidden = numero != 1` esconde a etapa 1 sempre que o número não for 1.
- O laço `for` percorre os 3 itens da barra de progresso e dá a classe `atual` (etapa em andamento), `feito` (anteriores) ou nenhuma. É um "se" dentro de outro "se" escrito com `? :`.
- `window.scrollTo(0, 0)` volta ao topo da página.

Eventos da etapa 1 (linhas 143 a 161)

```
143  pegar("tela-vaga").oninput = function () {  // qualquer mudança nos campos redesenha a tela
144    pegar("aviso").hidden = true;
145    atualizar();
146  };
147
148  pegar("tela-vaga").onsubmit = function (evento) {  // botão "Continuar"
149    evento.preventDefault();  // impede o formulário de recarregar a página
```

```
  const periodo = calcularPeriodo();
  if (periodo == null || vagaEscolhida == null) return;
  pegar("c-vaga").textContent = vagaEscolhida.codigo;
  pegar("c-tipo").textContent = vagaEscolhida.tipo;
  pegar("c-mod").textContent = ehDiaria() ? "Diária de 24 h" : "Por horário";
  pegar("c-entrada").textContent = dataHora(periodo.inicio);
  pegar("c-saida").textContent = dataHora(periodo.fim);
  pegar("c-total").textContent = moeda(periodo.valor);
  mostrarEtapa(2);
};

pegar("voltar").onclick = function () { mostrarEtapa(etapaAtual - 1); };
```

- `oninput`: roda quando o usuário muda qualquer campo do formulário. Esconde o aviso e chama `atualizar()`.
- `onsubmit` do formulário é o botão "Continuar". `evento.preventDefault()` **impede o comportamento padrão** do formulário (recarregar a página).
- Se o período ou a vaga não estão certos, `return` encerra a função. Se estão, copia os dados para o resumo da etapa 2 e chama `mostrarEtapa(2)`.
- O botão "Voltar" chama `mostrarEtapa(etapaAtual - 1)`.

mostrarErro() (linhas 163 a 168)

```
// ----- 5) Etapa 2: seus dados -----

function mostrarErro(mensagem) {
  pegar("erro-dados").textContent = mensagem;
  pegar("erro-dados").hidden = false;
}
```

- Escreve a mensagem de erro no parágrafo da etapa 2 e o torna visível.

Validação dos dados (etapa 2) (linhas 170 a 186)

```
pegar("form-dados").onsubmit = function (evento) {  // botão "Ir para pagamento"
  evento.preventDefault();
  const periodo = calcularPeriodo();
  const nome = pegar("nome").value.trim();                  // trim() tira os espaços das pontas
  const telefone = pegar("telefone").value.trim();
  const placa = pegar("placa").value.trim().toUpperCase();  // toUpperCase() deixa em maiúsculas

  // Confere cada campo. Se algo estiver errado, mostra o erro e para (return).
  if (!nome.includes(" ")) return mostrarErro("Digite o nome e o sobrenome.");
  if (isNaN(telefone) || telefone.length < 10 || telefone.length > 11) return mostrarErro("Telefone: só
números, com DDD (10 ou 11 dígitos).");
  if (placa.length != 7) return mostrarErro("A placa precisa ter 7 caracteres, como ABC1D23.");
  if (temConflito("placa", placa, periodo)) return mostrarErro("Essa placa já tem uma reserva nesse
período.");

  pegar("erro-dados").hidden = true;
  pegar("c-nome").textContent = nome + " · " + placa;
  mostrarEtapa(3);
};
```

- Lê os campos. `trim()` remove espaços do começo e do fim; `toUpperCase()` deixa a placa em maiúsculas.
- **Nome:** `nome.includes(" ")` confere se tem um espaço, ou seja, nome e sobrenome. O `!` significa "não".

- **Telefone:** `isNaN(telefone)` é verdadeiro se tiver letras ou símbolos. Depois confere o tamanho com `length`: 10 dígitos (fixo) ou 11 (celular), com DDD.
- **Placa:** precisa ter 7 caracteres (cabe o padrão antigo ABC1234 e o Mercosul ABC1D23).
- **Placa em uso:** `temConflito("placa", placa, periodo)` reaproveita a mesma regra de sobreposição.
- Cada `return mostrarErro(...)` mostra o erro e **para** a função. Se todos os testes passaram, esconde o erro e vai para a etapa 3.

Formas de pagamento (linhas 188 a 200)

```
188  // ----- 6) Etapa 3: pagamento -----
189
190  const textosPagamento = {  // a chave é o "value" de cada opção no HTML
191    cartao: "Cartão de crédito: você paga ao chegar ao estacionamento.",
192    "google-wallet": "Google Wallet: pagamento apenas simulado.",
193    pix: "Pix: veja o QR Code de demonstração (pagamento apenas simulado).",
194    dinheiro: "Dinheiro: você paga ao chegar ao estacionamento."
195  };
196
197  function pagamentoEscolhido() {  // devolve a opção marcada (ex.: "pix") ou null
198    const marcado = document.querySelector('input[name="pagamento"]:checked');
199    return marcado ? marcado.value : null;
200  }
```

- `textosPagamento` é um objeto: cada chave (cartao, pix...) é igual ao `value` da opção no HTML e guarda o texto que será mostrado. `"google-wallet"` precisa de aspas por ter hífen.
- `pagamentoEscolhido()` procura a opção marcada. Se achar, devolve o `value`; se não, devolve `null`.

Ao escolher o pagamento (linhas 202 a 208)

```
202  pegar("form-pagamento").onchange = function () {  // ao escolher uma forma: mostra o texto, o QR Code
(só no Pix) e libera o botão
203    const forma = pagamentoEscolhido();
204    pegar("pg-titulo").textContent = "Forma escolhida";
205    pegar("pg-texto").textContent = textosPagamento[forma];
206    pegar("pix").hidden = forma != "pix";
207    pegar("pagar").disabled = false;
208  };
```

- `onchange` roda quando o usuário escolhe uma opção: mostra o texto da forma escolhida, exibe o QR Code **só se for Pix** e libera o botão "Confirmar".
- Cartão e dinheiro são pagos no estacionamento; Google Wallet e Pix são apenas simulados. Nenhuma cobrança real é feita.

Confirmar: cria a reserva (linhas 210 a 244)

```
210  pegar("form-pagamento").onsubmit = function (evento) {  // botão "Confirmar reserva": cria a reserva
211    evento.preventDefault();
212    const periodo = calcularPeriodo();
213    if (periodo == null || temConflito("vaga", vagaEscolhida.codigo, periodo)) {  // confere de novo se a
vaga continua livre
214      mostrarEtapa(1);
215      atualizar();
216      return;
217    }
```

```
218
219   const reserva = {  // a reserva é um objeto com todos os dados
220     vaga: vagaEscolhida.codigo,
221     nome: pegar("nome").value.trim(),
222     placa: pegar("placa").value.trim().toUpperCase(),
223     pagamento: pagamentoEscolhido(),
224     inicio: periodo.inicio,
225     fim: periodo.fim,
226     valor: periodo.valor,
227     status: "ativa"  // depois pode virar "cancelada"
228   };
229   reservas.push(reserva);  // push coloca a reserva no final da lista
230   salvarReservas();
231
232   pegar("aviso").textContent = "Reserva da vaga " + reserva.vaga + " confirmada para " + reserva.nome +
". Total: " + moeda(reserva.valor) + ".";
233   pegar("aviso").hidden = false;
234   vagaEscolhida = null;  // limpa tudo para uma nova reserva
235   pegar("form-dados").reset();
236   pegar("form-pagamento").reset();
237   pegar("pix").hidden = true;
238   pegar("pagar").disabled = true;
239   pegar("pg-titulo").textContent = "Escolha uma opção acima";
240   pegar("pg-texto").textContent = "O total é o mesmo em todas as formas de pagamento.";
241   mostrarEtapa(1);
242   atualizar();
243   listarReservas();
244 };
```

- Antes de criar, **confere de novo** se a vaga continua livre. Se não estiver, volta à etapa 1.
- A reserva é um **objeto** com vaga, nome, placa, forma de pagamento, início, fim, valor e `status: "ativa"`.
- `reservas.push(reserva)` coloca a reserva no final da lista e `salvarReservas()` grava no navegador.
- Depois mostra a mensagem de sucesso, limpa a vaga escolhida e os formulários (`reset()`), volta à etapa 1 e redesenha a planta e a lista.

listarReservas(): desenha a lista (linhas 246 a 278)

```
246 // ----- 7) Minhas reservas -----
247
248 function listarReservas() {  // desenha a lista (a mais nova primeiro)
249   const lista = pegar("lista");
250   lista.innerHTML = "";
251   let ativas = 0;
252   if (reservas.length == 0) lista.textContent = "Nenhuma reserva por enquanto. Selecione uma vaga para
começar.";
253
254   for (let i = reservas.length - 1; i >= 0; i--) {
255     const reserva = reservas[i];
256     if (reserva.status == "ativa") ativas++;
257
258     const caixa = document.createElement("div");  // cada reserva vira uma caixa
259     caixa.className = "recibo";
260     const codigo = document.createElement("strong");
261     codigo.className = "codigo";
262     codigo.textContent = reserva.vaga;
263     const detalhes = document.createElement("p");
264     detalhes.textContent = reserva.nome + " · " + reserva.placa + " · " + dataHora(reserva.inicio) + "
até " +
265       dataHora(reserva.fim) + " · " + moeda(reserva.valor) + " · " + reserva.pagamento + " · " +
reserva.status;
266     caixa.append(codigo, detalhes);
```

```
267
268     if (reserva.status == "ativa") {  // só reservas ativas têm o botão de cancelar
269       const botao = document.createElement("button");
270       botao.className = "botao";
271       botao.textContent = "Cancelar reserva";
272       botao.onclick = function () { cancelarReserva(reserva); };
273       caixa.append(botao);
274     }
275     lista.append(caixa);
276   }
277   pegar("contador").textContent = ativas;
278 }
```

- Limpa a lista (`innerHTML = ""`) e, se não há reservas, mostra uma mensagem.
- O laço `for (let i = reservas.length - 1; i >= 0; i--)` anda **de trás para frente**, assim a reserva mais nova aparece primeiro.
- Conta as reservas ativas para o número ao lado do título.
- Cada reserva vira uma caixa criada com `createElement`: o código da vaga, um parágrafo com os detalhes e, **só se estiver ativa**, o botão "Cancelar reserva". Os textos são colocados com `textContent`, que é seguro: se alguém digitar código no nome, ele aparece como texto e não é executado.

cancelarReserva() (linhas 280 a 286)

```
280 function cancelarReserva(reserva) {  // a reserva deixa de estar "ativa", então a vaga volta a ficar
livre
281   if (!confirm("Cancelar a reserva da vaga " + reserva.vaga + "?")) return;
282   reserva.status = "cancelada";
283   salvarReservas();
284   atualizar();
285   listarReservas();
286 }
```

- `confirm()` abre uma caixa de "sim ou não". Se o usuário clicar em não, `return` encerra a função.
- Se confirmar, o status vira `"cancelada"`. Como `temConflito` só considera reservas "ativa", a vaga **volta a ficar livre**. Depois salva e redesenha planta e lista.

Tema e início do programa (linhas 288 a 303)

```
288 // ----- 8) Tema e início do programa -----
289
290 // O CSS troca as cores conforme o atributo data-tema ("claro" ou "escuro").
291 document.documentElement.dataset.tema = matchMedia("(prefers-color-scheme: dark)").matches ? "escuro" :
"claro";
292 pegar("tema").onclick = function () {
293   const raiz = document.documentElement;
294   raiz.dataset.tema = (raiz.dataset.tema == "escuro") ? "claro" : "escuro";
295 };
296
297 const hoje = new Date();  // a data já começa em hoje (formato AAAA-MM-DD), assim o período é válido
298 pegar("data").value = hoje.getFullYear() + "-" + String(hoje.getMonth() + 1).padStart(2, "0") + "-" +
String(hoje.getDate()).padStart(2, "0");
299 pegar("tarifas").textContent = "Valores de exemplo: " + moeda(VALOR_HORA) + " por hora ou " +
moeda(VALOR_DIARIA) + " pela diária.";
300
301 mostrarEtapa(1);   // começa na etapa 1
```

```
atualizar();       // desenha a planta e o resumo
listarReservas();  // mostra as reservas já salvas
```

- O tema é um atributo no elemento raiz: `document.documentElement.dataset.tema` vira `data-tema="claro"` ou `"escuro"`. O CSS troca as cores por causa dele (e mostra o ícone de sol ou lua).
- `matchMedia("(prefers-color-scheme: dark)").matches` pergunta ao computador se ele usa tema escuro. O site já abre combinando com o sistema.
- O botão de tema alterna entre "claro" e "escuro".
- A data de entrada já começa em **hoje** (`getMonth() + 1` porque os meses começam em 0; `padStart(2, "0")` coloca o zero à esquerda: 5 vira "05"). Assim o período é válido desde o início e a planta não aparece bloqueada.
- As três últimas linhas executam tudo pela primeira vez: `mostrarEtapa(1)`, `atualizar()` e `listarReservas()` (mostra as reservas já salvas).

Resumo para fechar a apresentação

- **Fluxo:** escolher período, escolher vaga, preencher dados, pagar, ver em "Minhas reservas".
- **Regras principais:** `calcularPeriodo()` calcula o valor; `temConflito()` impede reservas sobrepostas de vaga e de placa; `cancelarReserva()` libera a vaga de novo.
- **Dados:** tudo em um array `reservas` salvo em `localStorage` como JSON. Sem servidor.
- **Interface:** o JS só liga e desliga classes e atributos; o CSS decide o visual (inclusive o tema escuro, por variáveis).
- **Limites:** é um protótipo: pagamento simulado, dados só no navegador de quem usou.